

Project One

Taaruni Ananya

Georgia State University

Operating Systems (CSC 4320)

Dr. Roya Hosseini

September 30, 2026

Overview

This program is written in Java. It contains a text menu for easier use and four implemented algorithms: Round Robin, non-preemptive SJF, Best-Fit, and LRU. Round Robin uses a positive integer time quantum where arriving processes join the ready queue. In non-preemptive SJF, the process with the smallest burst time is selected among ready processes. In one algorithm, Best-Fit works for contiguous memory allocation where memory-block sizes and process memory request from a file, allocates requests in input order, and displays the selected block for each request. LRU is implemented in one algorithm for paging and page replacement where a positive frame count and page-reference string from a file are read.

The program is organized using a clear menu-driven interface to select the simulation, input file, and RR quantum. There are three main factors: readProcesses, readMemory, and readPaging. runSJF, runRR, resetProcesses, bestFit, and LRU are simulation. Finally, calculateMetrics is for calculations and printResults is for output.

Implementation Details

Items are not sorted and run at once, the process is more regulated. SJF will maintain a clock where each time the CPU is available and free, it will scan through all unfinished processes. If there are any that have a higher arrival time than the clock, they will be ignored and the best remaining one will be selected. Round Robin will use an ArrayDeque for the ready queue. For the feed, the process list is sorted by arrival time and a pointer is used to implement a process into the queue when its arrival time is less than or equal to that of the clock's. If the

ready queue or SJF is empty, the clock will then jump to the next arrival and record the gap as idle.

SJF for tie-breaking has a candidate that replaces the current best if its burst time is smaller. Another case is when its burst time is equal and its arrival time is earlier than others. In the case that both are equal, the process that arrived earlier is selected.

When a process is finished, the completion time is stored. The turnaround time (TAT) is completion time - arrival time and waiting time (WT) is turnaround time - burst time. CPU Utilization is $\text{CPU busy time} / \text{total elapsed simulation time} \times 100$. Measure elapsed time from time 0 through the final completion, so initial and intermediate idle periods count.

Round Robin requeues in the following steps. A process first runs for the minimum of quantum and remaining ($\min(\text{time}, p.\text{remainingTime})$) and when the slice ends, any process that arrived by this time will be added to the queue first. After this is completed, the unfinished process will be added to the back of the queue. Any processes without time left will get their completion time recorded.

For contiguous memory allocation with Best-Fit, requests are handled in file order. All blocks are scanned by the program for every request. The smallest block that is large enough is remembered. Then, this block is reduced by the request size. If there aren't any blocks large enough to satisfy the request, it is reported as unallocated. Any remaining blocks will have their sizes entered at the end.

LRU for paging and page replacement has frames that are fixed slots with a time stamp for when they are last used. If a frame already has a reference, its time stamp is refreshed. If there aren't any empty frames, the one with the smallest time stamp is replaced. After all frame contents the references are printed, then totals, hit ratio, and fault ratio.

Results

<pre> 1) Round Robin 2) SJF 3) Compare RR vs SJF 4) Best-Fit memory 5) LRU paging 0) Exit Choose: 1 Process file: scheduled_roundrobin.txt Time quantum: 3 PID Arrival Burst CT TAT WT P1 0 8 23 23 15 P2 1 5 17 16 11 P3 2 9 29 27 18 P4 3 4 21 18 14 P5 20 3 6 6 3 Average WT : 12.20 Average TAT : 18.00 CPU Utilization : 100.00%</pre>	<pre> 1) Round Robin 2) SJF 3) Compare RR vs SJF 4) Best-Fit memory 5) LRU paging 0) Exit Choose: 1 Process file: scheduled_roundrobin.txt Time quantum: 5 PID Arrival Burst CT TAT WT P1 0 8 22 22 14 P2 1 5 10 9 4 P3 2 9 26 24 15 P4 3 4 19 16 12 P5 20 3 29 9 6 Average WT : 10.20 Average TAT : 16.00 CPU Utilization : 100.00%</pre>	<pre> 1) Round Robin 2) SJF 3) Compare RR vs SJF 4) Best-Fit memory 5) LRU paging 0) Exit Choose: 2 Process file: scheduled_roundrobin.txt PID Arrival Burst CT TAT WT P1 0 8 8 8 0 P2 1 5 17 16 11 P3 2 9 26 24 15 P4 3 4 12 9 5 P5 20 3 29 9 6 Average WT : 7.40 Average TAT : 13.20 CPU Utilization : 100.00%</pre>
---	---	--

Comparing these results, SJF has a lower average WT of 7.40 and TAT of 13.20. First, when P1 finishes at 8, SJF then proceeds to run the shortest job that is ready, in this case P4, P2, and then P3. Here, the shorter jobs aren't waiting for the longer ones to complete. Round Robin is where the CPU is used and shared, this leads to the longer jobs to get interrupted and finish later. Every process here waits for other processes' slices. In the second screenshot, there are fewer interruptions that lead to a faster and better outcome. However, the average WT and TAT are still lesser than that of SJF. As for utilization, it remains the same across all three pictured cases because there are always processes ready. This is possible because utilization depends on arrival pattern rather than the algorithm.

Other results:

```

1) Round Robin  2) SJF  3) Compare RR vs SJF
4) Best-Fit memory  5) LRU paging  0) Exit
Choose: 4
Memory file: mem.txt

=== Contiguous Memory Allocation: Best-Fit ===
Initial blocks: [100, 500, 200, 300, 600]
Request 1 (700) -> UNALLOCATED
Request 2 (417) -> Block 2 (500 -> 83 left)
Request 3 (112) -> Block 3 (200 -> 88 left)
Request 4 (426) -> Block 5 (600 -> 174 left)
Request 5 (800) -> UNALLOCATED
Remaining block sizes: [100, 83, 88, 300, 174]

1) Round Robin  2) SJF  3) Compare RR vs SJF
4) Best-Fit memory  5) LRU paging  0) Exit
Choose: 5
Paging file: paging.txt

=== Paging: LRU (3 frames) ===
7 [7 - -] FAULT
0 [7 0 -] FAULT
1 [7 0 1] FAULT
2 [2 0 1] FAULT
0 [2 0 1] HIT
3 [2 0 3] FAULT
0 [2 0 3] HIT
4 [4 0 3] FAULT

Total references: 8
Hits : 2
Faults : 6
Hit ratio : 0.25
Fault ratio : 0.75

```

Testing

All use cases have input files and have saved outputs. Initially, values for testing were hardcoded and then changed to work with files.

Idle time is tested with `scheduled_idle.txt` where it is verified with P1 and P2 finishing at 5, and P3 arriving at 10. Both algorithms show 5 to 10 as an idle gap. Tie is tested with `scheduled_tie.txt` where it is verified that at time 6, P2, P3, and P4 have burst 4. SJF runs P2 through P4. P2 and P3 have an arrival and the file order selects to push P4 because it arrived later. Round Robin cycles and requeue are tested with `scheduled_roundrobin.txt` where P1 appears thrice with quantum 3 and P5 arrives at 20 when a slice ends. This is a success because it runs before P3 which is requeued. Allocation failure is tested with `mem.txt` where the 700 and 800 requests are larger and the output is printed as UNALLOCATED. 417 is moved to the 500 block, 112 to the 200 block, and 426 to the 600 block. Page hits and faults are tested with `paging.txt` where there are 3 frames and 8 references with 2 hits and 6 faults. Page 0's hit refreshes the use and page 1 is replaced when page 3 arrives.

Challenges and Solutions

One challenge encountered was with the clock and idle time. The clock did not move forward once a process ran and the later processes dealt with being treated as they arrived to an empty CPU. This was fixed by updating the clock after every run and saving the idle start before jumps. Another small but significant challenge was where the slice length was inputted from the clock. This caused an issue where there were slices with no length and an endless loop. Catching this bug and renaming it to proper terms fixed this issue. A third challenge was with ordering new arrivals and unfinished processes entering the queue. The problem was solved by manually tracing examples and using another case to test.

AI-Use Disclosure

An AI assistant was used to plan the project's structure before beginning the assignment. I was able to make a simple plan on how to implement and organize the files to work. I also used GitHub Copilot to debug and clean up formatting issues.